

UkrAid

Global Solution 2026 - 1° Semestre



1. O Problema e a Solução

O UkrAid é uma CLI (Command Line Interface) desenvolvida como solução para a Global Solution do 1º Semestre de 2026. O sistema procura centralizar dados de inteligência de fontes abertas (OSINT) para auxiliar civis em zonas de conflito.

O programa inclui um sistema de avaliação de risco baseado na distância do usuário, um monitoramento de alertas de ataques aéreos em tempo real, a localização de recursos humanitários dentro de zonas de conflito e o rastreamento de eventos de conflitos (com delay de 12 meses).

A conexão da solução UkrAid com a indústria espacial se dá com o uso de dados derivados de sensoriamento remoto e inteligência orbital. A API ACLED (Armed Conflict Location & Event Data) agrega eventos documentados com base em imagens de satélite e relatórios georreferenciados.

O problema central abordado foi: **“como quantificar matematicamente o risco que um civil corre em relação à distância de um evento?”** A resposta: Um modelo que capture a atenuação física da energia destrutiva quanto o comportamento assintótico do risco em longas distâncias. Para isso, a lei do Inverso do Quadrado foi adotada como modelo matemático central.

2. Fórmula Haversine | Distância Geodésica

A Terra é esférica e calcular distâncias pela diferença entre coordenadas introduz erros de forma crescente conforme as distâncias aumentam, algo inaceitável em um sistema de alerta de riscos, onde possivelmente, vidas estariam em risco.

A Fórmula Haversine resolve isso calculando a distância ao longo do *arco de grande círculo* (o menor caminho entre X e Y na superfície da terra).

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right)$$

Variável	Descrição
d	Distância geodésica entre dois pontos
r	Raio médio da Terra (6371 km)
Δ	Diferença entre (Δ ou ϕ)

λ	Longitudes (1 ou 2)
φ	Latitudes (1 ou 2)

3. Modelo Matemático

3.1 Construção do Modelo

O modelo de risco foi construído pela combinação de dois princípios físicos complementares, ambos fundamentados em fenômenos estudados na Indústria Espacial:

Lei do Inverso do Quadrado ($1/d^2$): Governa a propagação de energia irradiada.

A intensidade decai com o quadrado da distância.

Atenuação Exponencial (e^{-kd}): Modela absorção pelo meio físico.

A função combinada tem como objetivo calcular a dispersão de energia (dano/risco) que uma explosão possa ter em relação a distância.

$$R(d) = \frac{e^{-k \cdot d}}{d^2}$$

3.2 Domínio, Imagem e Comportamento

Propriedade	Valor	Interpretação
Domínio	d	d = 0 é indefinido ($1/0^2$). Distância negativa não existe.
Imagem	R(d)	O risco nunca é zero.
Se d = 0	R(d) = Infinito	No epicentro o risco tende ao infinito, ou destruição total.
Se d = Infinito	R(d) = 0	O risco diminui porém nunca se anula.

3.3 Constante K Relativo ao Tipo de Ameaça

A constante k calibra a velocidade de dissipação do risco, ou seja, cada tipo de ameaça tem seu próprio valor k .

Tipo de Ameaça	k	Justificativa
Tropa em movimento (Batalhas)	0.001	Ameaça móvel e persistente, perigo se mantém em grandes distâncias.
Ataque de Míssil (Bombardeamento)	0.005	Cobertura ampla porém temporária.

4. Implementação Computacional em Python

O modelo matemático foi implementado no módulo **risk_model.py**, que foi importado para o **main.py** onde a solução está localizada. O módulo é chamado para cada evento retornado pela API ACLED, calculando o risco com dados georreferenciados reais.

```

1  #Imports:
2  import math
3
4  #K_ Consts:
5  # Threat attenuation constants
6  K_TROOPS = 0.001      # persistent, mobile threat
7  K_MISSILE_MAX = 0.005 # highest risk per distance unit
8
9  #Functions:
10 def haversine(lat1, lon1, lat2, lon2):
11     r = 6371 #This is earth's radius in km
12     phi1, phi2 = math.radians(lat1), math.radians(lat2)
13     dphi = math.radians(lat2 - lat1)
14     dlamba = math.radians(lon2 - lon1)
15
16     a = math.sin(dphi/2)**2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlamba/2)**2
17     return 2 * r * math.asin(math.sqrt(a))
18
19
20 def troop_risk(d, k = K_TROOPS):
21     return math.exp(-k * d) / (d ** 2)
22
23
24 def missile_strike(d, k = K_MISSILE_MAX):
25     return math.exp(-k * d) / (d ** 2)
26
27
28 def risk_label(risk):
29     if risk > 0.001:
30         return f"[⚠️ ALERT ⚠️] Critical risk of: {risk:.4f}"
31     if risk > 0.0001:
32         return f"[⚠️ ALERT] High risk of :{risk:.4f}"
33     if risk > 0.00001:
34         return f"Moderate risk of :{risk:.4f}"
35     else:
36         return "Low Distance-Based Risk"

```